

TPM Incident Orchestrator Copilot

Executive Summary of the AI-Assisted Incident Management Prototype

PROTOTYPE STATUS	DEMO OUTCOME	PRIMARY USER
Successfully executed	Incident resolved	Technical Program Manager

Executive Overview

The prototype demonstrated how an AI agent can support a Technical Program Manager during and after a production incident without acting as a hands-on debugger or autonomous production operator. In the simulated Sev-1 scenario, customers could not place orders. The agent organized the response, established ownership, generated stakeholder updates, documented the engineering-owned decision to restart the Order Processing Service, tracked recovery, and created follow-up actions.

Business result: the service restart restored order placement, while the TPM agent maintained coordination, communication, decision visibility, and post-incident accountability.

What the Prototype Demonstrated

- **Incident organization:** moved the event through investigation, monitoring, and resolution states.
- **Ownership visibility:** assigned investigation streams and follow-up actions to accountable roles.
- **Decision governance:** recorded the service restart as an engineering-owned and approved action.
- **Business translation:** transformed confirmed technical updates into customer and business language.
- **Stakeholder communication:** generated structured updates before and after recovery.
- **Post-incident accountability:** created corrective actions for engineering, SRE, the service owner, and the TPM.

Operating Boundary

The agent did not diagnose root cause, analyze logs, restart the service, approve production changes, or make engineering decisions. The authorized engineering lead retained responsibility for selecting, approving, and executing the recovery action.

Execution Summary

Stage	Agent contribution	Human authority
Activation	Created coordination checklist, timeline, ownership structure, and communication cadence.	TPM confirms roles and response model.
Investigation	Assigned service, platform, and customer-impact workstreams.	Engineering teams perform technical investigation.
Decision	Recorded the restart decision, rationale, owner, and approval.	Authorized engineering lead selects and approves mitigation.
Recovery	Changed status to monitoring and generated recovery communication.	Engineering validates service health and stability.
Closure	Recorded resolution and created corrective follow-up actions.	TPM and technical owner confirm closure criteria.

MCP Integration Readiness

The implementation established a common placeholder interface for future context providers. All integrations correctly reported that they were not connected, and no external systems were queried or changed.

Placeholder	Intended context
Jira	Releases, milestones, dependencies, delivery risk
ServiceNow	Incidents, changes, problem-management context
Confluence	Runbooks, ownership, previous incidents
PagerDuty	On-call and escalation information
Teams	Communications, decisions, action-item extraction
Datadog / Splunk	Service-health and technical context summaries
GitHub / Deployment	Deployment awareness and release correlation

Post-Incident Accountability

- Determine why the service became unresponsive.
- Review service-health monitoring and alert coverage.
- Update the recovery runbook.
- Facilitate the post-incident review and track actions through closure.

Production-Readiness Assessment

The prototype is suitable as a proof of concept. Before production use, the following gaps must be addressed.

Priority area	Required improvement
Live integrations	Replace placeholders with authenticated MCP clients and configured servers.
Approvals and authority	Require human approval before communications, record updates, incident resolution, or operational actions.
State consistency	Separate historical impact, current impact, recovery state, and residual risk to prevent conflicting updates.
Closure controls	Block closure when investigations remain open or actions lack owners, dates, and closure criteria.
Evidence and auditability	Attach source system, record ID, retrieval time, and verification status to retrieved facts.
Security and reliability	Add role-based access, secret management, audit logs, persistence, retries, monitoring, and failure isolation.

Overall Assessment

The prototype successfully demonstrated a TPM-focused incident-management model. Engineering remained responsible for service restoration, while the agent supported the TPM by maintaining organizational control, communication quality, decision visibility, delivery awareness, and corrective-action accountability.

The workflow is strong enough for a proof-of-concept demonstration, but production deployment requires live MCP integrations, consistent state management, enterprise approvals, closure validation, and security governance.

Engineers restore the system. The TPM Incident Orchestrator Copilot helps the TPM restore alignment, maintain trust, protect delivery commitments, and ensure corrective actions are completed.

Recommendation

Proceed with a controlled proof of concept focused on read-only context retrieval, TPM-approved communications, explicit source attribution, and measurable follow-up closure. Defer autonomous production actions until governance, authorization, and audit controls are established.